How to Validate v1d JSON Artifacts

(Version D — Governance Stress-Testing and Ethical Evaluation)

Purpose

This document explains how to validate the **Version D companion JSON artifacts** using the provided Python script `validate_jsonschema.py`.

Validation confirms that JSON files **structurally conform** to the published schemas.
It does **not** imply approval, certification, compliance, trust, or ethical sufficiency.

This process is **informative only** and supports inspection and standards discussion.

What Is Being Validated

Two types of JSON artifacts are included with Version D:

1. **Scenario Index**
   - `v1d_scenarios.index.json`
   - Validated against: `v1d_scenarios.schema.json`
2. **Stress-Test Evidence Record**
   - `apod.v1d.stress_evidence_record.example.json`
   - Validated against: `apod.v1d.stress_evidence_record.schema.json`

Prerequisites

- **Python 3.9 or later**, available as `py`
- **PowerShell 7** (recommended)
- Internet access (for one-time dependency installation)

Repository Layout (Important)

The validation script assumes the following **repository structure**:

```
v1d-Governance Stress-Testing and Ethical Evaluation/
├─ tools/
│  └─ validate_jsonschema.py
├─ companions/
│  └─ schemas/
│     ├─ v1d_scenarios.schema.json
│     └─ apod.v1d.stress_evidence_record.schema.json
└─ examples/
   ├─ v1d_scenarios.index.json
   └─ apod.v1d.stress_evidence_record.example.json
```

Paths used in the commands below are **relative to the repository root**.
The script automatically resolves paths correctly regardless of the current working directory.

Step 1 — Install Dependencies (One-Time)

Open **PowerShell 7** and run:

```
py -m pip install --upgrade jsonschema referencing
```

You only need to run this once per Python environment.

Note:
You may see a message indicating a newer version of `pip` is available.
This notice is informational and does **not** affect validation.

Step 2 — Navigate to the Script Location

Change directory to the `tools` folder:

```
cd "C:\v1d-Governance Stress-Testing and Ethical Evaluation\tools"
```

Confirm the script is present:

```
dir validate_jsonschema.py
```

Step 3 — Validate the Scenario Index

Run the following command:

```
py validate_jsonschema.py companions\schemas\v1d_scenarios.schema.json `
   examples\v1d_scenarios.index.json
```

Expected output (success)

```
VALIDATION: PASS
Schema : C:\v1d-Governance Stress-Testing and Ethical
Evaluation\companions\schemas\v1d_scenarios.schema.json
Data   : C:\v1d-Governance Stress-Testing and Ethical
Evaluation\examples\v1d_scenarios.index.json
```

Step 4 — Validate a Stress-Test Evidence Record

Run:

```
py validate_jsonschema.py
companions\schemas\apod.v1d.stress_evidence_record.schema.json `
   examples\apod.v1d.stress_evidence_record.example.json
```

Expected output (success)

```
VALIDATION: PASS
Schema : C:\v1d-Governance Stress-Testing and Ethical
Evaluation\companions\schemas\apod.v1d.stress_evidence_record.schema.json
Data   : C:\v1d-Governance Stress-Testing and Ethical
Evaluation\examples\apod.v1d.stress_evidence_record.example.json
```

Interpreting Validation Results

- **PASS**
  The JSON file conforms structurally to the schema.
- **FAIL**
  The script will report:
  - the JSON path where the error occurred,
  - a human-readable explanation, and
  - the schema rule that was violated.

Validation checks **structure and allowed values only**.
It does **not** evaluate governance decisions, ethical reasoning, or trust posture.

Common Errors and Fixes

“Schema file not found” or “Data file not found”

- Ensure you are in the `tools` directory.
- Ensure the repository layout matches the structure shown above.

“Invalid JSON … Expecting value: line 1 column 1”

- The JSON file may contain comments or non-JSON content.
- JSON does **not** support comments (`//` or `/* */`).

Scope and Limitations

This validation process:

- ✓ supports inspectable, repeatable review
- ✓ confirms schema alignment
- ✓ is suitable for standards and regulatory discussion

It does **not**:

- certify systems,
- determine compliance,
- assign scores,
- approve deployment,
- or elevate trust.

Summary

The `validate_jsonschema.py` script provides a **lightweight structural validation step** for Version D companion artifacts. It ensures consistency and inspectability without introducing tooling dependency, certification semantics, or normative claims.